Python Programming
Unit 06 – Lecture 04 Notes
Missing Data, Aggregation, Combining, CSV I/O

Tofik Ali

February 17, 2026

Contents

1 Lecture Overview **1**

2 Core Concepts **1**
2.1 Missing Values 1
2.2 Aggregation with `groupby` 2
2.3 Combining DataFrames 2
2.4 CSV Import/Export 2

3 Demo Walkthrough **2**

4 Interactive Checkpoints (with Solutions) **2**

5 Practice Exercises (with Solutions) **3**

6 Exit Question (with Solution) **3**

1 Lecture Overview

Real-world datasets are rarely perfect. Common issues:

- missing values,
- inconsistent categories,
- multiple tables that must be combined,
- and reading/writing data files (CSV).

This lecture teaches practical Pandas tools to handle these issues.

2 Core Concepts

2.1 Missing Values

Pandas represents missing numeric values as `NaN`. Useful functions:

- `df.isna()` and `df.notna()`
- `df.dropna()` remove rows/columns with missing data
- `df.fillna(value)` fill missing values

```
print(df.isna().sum())
df["marks"] = df["marks"].fillna(df["marks"].mean())
```

Important: filling strategy depends on context. Replacing missing marks with 0 may be wrong if the mark is unknown (not actually 0).

2.2 Aggregation with `groupby`

`groupby` groups rows by a category and then applies a function:

```
avg = df.groupby("city")["marks"].mean()
count = df.groupby("city")["sapid"].count()
```

2.3 Combining DataFrames

`concat` stacks data:

```
combined = pd.concat([df1, df2], ignore_index=True)
```

`merge` joins data like SQL:

```
merged = pd.merge(df, city_state, on="city", how="left")
```

2.4 CSV Import/Export

```
df = pd.read_csv("data/student_scores.csv")
df.to_csv("data/cleaned.csv", index=False)
```

3 Demo Walkthrough

Data: `data/student_scores.csv`
Script: `demo/pandas_missing_data_demo.py`

The demo:

- prints missing-value counts,
- fills missing `city` and `marks`,
- computes average marks per city,
- merges a city-to-state mapping,
- exports a cleaned CSV.

4 Interactive Checkpoints (with Solutions)

Checkpoint 1 Solution

Question: what does `fillna(0)` do?
Answer: it replaces missing values (`NaN`) with 0 in the selected Series/DataFrame.

Checkpoint 2 Solution

Question: when use `merge` instead of `concat`?
Answer:

- Use `merge` when you want to join tables using a key (like `city`).
- Use `concat` when you want to stack rows/columns.

5 Practice Exercises (with Solutions)

Exercise 1: Fill Missing Marks with Median

Solution:

```
median = df["marks"].median()
df["marks"] = df["marks"].fillna(median)
```

Exercise 2: City-wise Count

Solution:

```
print(df.groupby("city")["sapid"].count())
```

6 Exit Question (with Solution)

Question: read `"marks.csv"` into a DataFrame?
Answer:

```
df = pd.read_csv("marks.csv")
```

Appendix: Slide Deck Content (Reference)

The material below is a reference copy of the slide deck content. Exercise solutions are explained in the main notes where applicable.

Title Slide

Repository: `https://github.com/tali7c/Python-Programming`

Quick Links

Core Concepts **Demo** **Interactive** **Summary**

Agenda

- Core Concepts
- Demo
- Interactive
- Summary

Learning Outcomes

- Detect and handle missing values (`NaN`)
- Use aggregation (`groupby`) for summaries
- Combine DataFrames (`concat`, `merge`)
- Import/export CSV using `read_csv` and `to_csv`

Missing Data

- Missing values appear as `NaN` in Pandas
- Useful functions:
  - `isna()`, `notna()`
  - `dropna()`
  - `fillna(value)`

Aggregation with `groupby`

```
avg_by_city = df.groupby("city")["marks"].mean()
```

- Useful for summaries (average marks per city, counts per category)

Combining DataFrames

- `pd.concat([...])`: stack rows or columns
- `pd.merge(a, b, on="key")`: SQL-like join

CSV Import/Export

```
df = pd.read_csv("data/student_scores.csv")
df.to_csv("data/cleaned.csv", index=False)
```

Demo: Missing Data + Groupby + CSV

- Data: `data/student_scores.csv`
- Script: `demo/pandas_missing_data_demo.py`
- Shows:
  - reading CSV
  - missing value handling
  - groupby aggregation
  - merge with another DataFrame

Checkpoint 1

Question: What does `fillna(0)` do?

Checkpoint 2

Question: When would you use `merge` instead of `concat`?

Think-Pair-Share

Discuss:

- Is it always correct to replace missing marks with 0? Why/why not?

Key Takeaways

- Missing values can be detected and filled/dropped
- `groupby` provides powerful summaries
- `merge` joins tables using keys; `concat` stacks data
- CSV I/O connects Pandas with real-world datasets

Exit Question

Write one line to read a CSV file named `"marks.csv"` into a DataFrame.